



INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	Tema de la guía práctica proporcionada por el docente
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	Primero "B"
Alumnos participantes:	Muyulema Moyolema Mateo Alexander
Asignatura:	Algoritmos y Lógica de Programación
Docente:	Ing José Rubén Caiza

II. INFORME DE GUÍA PRÁCTICA

1.1 Objetivos

General:

Desarrollar un sistema básico de control de estudiantes y calificaciones aplicando el paradigma de la Programación Orientada a Objetos (POO) en los lenguajes C++ y Java, para gestionar de forma automatizada y estructurada el registro, procesamiento de notas, cálculo de promedios y determinación del estado académico de los alumnos en la asignatura de Algoritmos y Lógica de Programación.

Específicos:

- Diseñar e implementar la clase Estudiante respetando el principio de encapsulamiento mediante el uso de atributos privados y sus respectivos métodos de acceso (getters y setters) en ambos lenguajes de programación.
- Desarrollar mecanismos de control y validación de flujos de entrada que restrinjan el ingreso de calificaciones estrictamente al rango numérico de 0 a 10, evitando inconsistencias o fallos lógicos durante la ejecución del programa.
- Generar reportes consolidados y estadísticos cuantitativos al finalizar el procesamiento de un mínimo de 5 estudiantes, detallando la nómina completa, promedios individuales y la contabilidad exacta de estudiantes en estado "Aprobado" y "Reprobado".

1.2 Modalidad

Presencial

1.3 Tiempo de duración

Presenciales: 6

No presenciales: 0

1.4 Instrucciones

- La tarea se debe realizar de forma individual.
- Lea detenidamente cada uno de los ejercicios planteados en el aula virtual y desarrolle lo solicitado.
- Codifique en Java cada uno de los ejercicios planteados.

1.5 Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

- ☒ Plataformas educativas
- ☐ Simuladores y laboratorios virtuales
- ☒ Aplicaciones educativas
- ☐ Recursos audiovisuales
- ☐ Gamificación
- ☒ Inteligencia Artificial



Otros (Especifique): _____

1.6 Actividades por desarrollar

1. Análisis del Problema

El proyecto consiste en desarrollar un sistema informático para gestionar las calificaciones de un grupo de estudiantes aplicando la Programación Orientada a Objetos (POO). Para cumplir con esto, se abordan tres puntos clave:

- **Organización y protección de datos (Encapsulamiento):** En lugar de tener nombres, cédulas y notas guardados en variables sueltas y desordenadas, el problema se resuelve creando una clase llamada Estudiante. Esta clase agrupa todos los datos de un alumno y los protege con acceso privado, controlando la información de forma ordenada y segura a través de métodos de acceso (getters y setters).
- **Cálculos automáticos:** Calcular manualmente los promedios y definir si un alumno aprueba o reprueba quita tiempo y puede generar errores. Para solucionar esto, el sistema incluye las funciones internas calcularPromedio() y determinarEstado(). De esta manera, cada vez que se registra o modifica una nota, el programa calcula el promedio automáticamente y define si el estudiante está "Aprobado" o "Reprobado" según la nota base de 7.00.
- **Control de errores al ingresar datos (Validación):** Si el usuario ingresa por accidente notas inválidas (como un 15 o un número negativo) o una cédula con más o menos de 10 dígitos, el programa podría fallar o procesar datos falsos. El problema se soluciona programando funciones auxiliares que revisan que las notas estén estrictamente entre 0 y 10, y que la cédula tenga exactamente los 10 números reglamentarios, bloqueando cualquier ingreso incorrecto hasta que se escriba bien.

2. Variables de Entrada, Proceso y Salida

Entrada

- cedula (Cadena de caracteres): Almacena de forma temporal o definitiva el número de identificación del estudiante para pasar por los filtros de comprobación de longitud.
- nombre y apellido (Cadena de caracteres): Registran las identidades alfabéticas individuales del alumno capturadas desde el teclado.
- nota1, nota2, nota3 (Decimales): Calificaciones numéricas provistas por el operador que representan el rendimiento del estudiante en cada uno de los tres periodos evaluativos.

Proceso

- promedio (Decimal): Variable interna encargada de almacenar el resultado aritmético del cálculo: $\text{promedio} = (\text{nota1} + \text{nota2} + \text{nota3}) / 3.0$
- estado (Cadena de caracteres): Variable de control lógico que define la condición académica final tomando las cadenas "Aprobado" o "Reprobado".
- listaEstudiantes (Arreglo dinámico): Contenedor estructurado elástico encargado de agrupar y preservar los múltiples objetos creados de tipo Estudiante.
- acumuladorAprobados y acumuladorReprobados (Enteros): Variables contadoras aritméticas que se incrementan progresivamente de uno en uno para generar las estadísticas globales finales.

Salida

- El programa genera un reporte consolidado en la pantalla de comandos estructurado a través de una interfaz tabular ordenada. Imprime una matriz de texto que alinea perfectamente los campos de Cedula, Nombre, Apellido, las tres calificaciones validadas, el Promedio final con precisión de dos decimales y el Estado resultante.
- Despliega al final de la ejecución el resumen numérico cuantitativo que muestra el total de alumnos aprobados y reprobados obtenidos en la muestra de datos.

3. Algoritmo

1. Inicio.
2. Definir la estructura de la clase Estudiante especificando sus propiedades privadas y sus funciones públicas de lectura y escritura (getters y setters).
3. Instanciar un arreglo dinámico para almacenar colecciones de objetos del tipo Estudiante.



4. Establecer un bucle de control para repetir el ciclo de captura de datos un mínimo de 5 veces.
5. Dentro del bucle de captura:
 - A. Solicitar el número de cédula del alumno.
 - B. Validar que la cédula contenga de forma estricta 10 dígitos numéricos y que los dos primeros caracteres representen una provincia válida. Si no cumple, limpiar el error y volver a solicitarla.
 - C. Capturar el nombre y apellido del estudiante.
 - D. Solicitar la Nota 1, Nota 2 y Nota 3 de manera consecutiva.
 - E. Evaluar cada nota mediante un bucle de reintento: si el valor ingresado es menor a 0 o mayor a 10 (o si se ingresan letras), reportar el fallo y bloquear el flujo hasta recibir un número válido.
 - F. Instanciar el objeto Estudiante enviar los parámetros recolectados a su constructor.
 - G. Añadir de forma secuencial el objeto creado al arreglo dinámico.
6. Imprimir las líneas de cabecera que delimitan visualmente la tabla de reporte en la consola.
7. Inicializar los contadores de rendimiento académico global en cero.
8. Iniciar un recorrido iterativo sobre cada elemento de la lista de estudiantes:
 - A. Invocar el método interno mostrarInformacion() para imprimir en una fila tabulada todos los campos calculados del alumno.
 - B. Evaluar si la propiedad de estado del objeto actual equivale al texto "Aprobado".
 - C. Si se cumple: Incrementar el contador de aprobados en una unidad.
 - D. Si no se cumple: Incrementar el contador de reprobados en una unidad.
9. Desplegar en pantalla los valores finales de ambos contadores cuantitativos.
10. Fin.

4.Pseudocódigo

Algoritmo Control_Calificaciones_Estudiantes

// Definición de variables globales para el sistema

Definir limiteEstudiantes, i Como Entero;

limiteEstudiantes <- 5;

// Dimensionamiento simulación de objetos en memoria paralela

Definir cedulas, nombres, apellidos, estados Como Caracter;

Definir notas1, notas2, notas3, promedios Como Real;

Dimension cedulas[5], nombres[5], apellidos[5], estados[5];

Dimension notas1[5], notas2[5], notas3[5], promedios[5];

Escribir "=== SISTEMA DE CONTROL DE ESTUDIANTES Y CALIFICACIONES
===";

Escribir "Asignatura: Algoritmos y Logica de Programacion";

Escribir "";

// Proceso de captura estructurada y validada

Para i <- 1 Hasta limiteEstudiantes Con Paso 1 Hacer

Escribir "--- Registro del Estudiante ", i, " ---";

// Validación de longitud y formato de cédula básica

Definir cedulaIngresada Como Caracter;

Definir cedulaValida Como Logico;

cedulaValida <- Falso;

Mientras cedulaValida = Falso Hacer

Escribir "Ingrese Cedula (10 digitos): ";



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
Leer cedulaIngresada;
Si Longitud(cedulaIngresada) = 10 Entonces
    cedulas[i] <- cedulaIngresada;
    cedulaValida <- Verdadero;
Sino
    Escribir "[Error] Formato incorrecto. Ingrese exactamente 10 numeros.";
FinSi
FinMientras

Escribir "Ingrese Nombre: ";
Leer nombres[i];
Escribir "Ingrese Apellido: ";
Leer apellidos[i];

// Validación de rangos para la Nota 1
Definir notaIngresada Como Real;
notaIngresada <- -1;
Mientras notaIngresada < 0 O notaIngresada > 10 Hacer
    Escribir "Ingrese Nota 1 (0-10): ";
    Leer notaIngresada;
    Si notaIngresada >= 0 Y notaIngresada <= 10 Entonces
        notas1[i] <- notaIngresada;
    Sino
        Escribir "[Error] Rango no valido. Intente nuevamente.";
    FinSi
FinMientras

// Validación de rangos para la Nota 2
notaIngresada <- -1;
Mientras notaIngresada < 0 O notaIngresada > 10 Hacer
    Escribir "Ingrese Nota 2 (0-10): ";
    Leer notaIngresada;
    Si notaIngresada >= 0 Y notaIngresada <= 10 Entonces
        notas2[i] <- notaIngresada;
    Sino
        Escribir "[Error] Rango no valido. Intente nuevamente.";
    FinSi
FinMientras

// Validación de rangos para la Nota 3
notaIngresada <- -1;
Mientras notaIngresada < 0 O notaIngresada > 10 Hacer
    Escribir "Ingrese Nota 3 (0-10): ";
    Leer notaIngresada;
    Si notaIngresada >= 0 Y notaIngresada <= 10 Entonces
        notas3[i] <- notaIngresada;
    Sino
        Escribir "[Error] Rango no valido. Intente nuevamente.";
    FinSi
FinMientras

// Procesamiento automatizado interno del Estudiante
promedios[i] <- (notas1[i] + notas2[i] + notas3[i]) / 3.0;

Si promedios[i] >= 7.00 Entonces
```



```
    estados[i] <- "Aprobado";
Sino
    estados[i] <- "Reprobado";
FinSi
Escribir "";
FinPara

// Despliegue de los resultados obtenidos en formato de reporte
Escribir "===== LISTADO DE
ESTUDIANTES =====";
Escribir "Cedula    Nombre    Apellido    Nota 1 Nota 2 Nota 3 Promedio Estado";
Escribir "-----";
-----";

Definir aprobados, reprobados Como Entero;
aprobados <- 0;
reprobados <- 0;

Para i <- 1 Hasta limiteEstudiantes Con Paso 1 Hacer
    Escribir cedula[i], " ", nombres[i], " ", apellidos[i], " ", notas1[i], " ",
    notas2[i], " ", notas3[i], " ", promedios[i], " ", estados[i];

    Si estados[i] = "Aprobado" Entonces
        aprobados <- aprobados + 1;
    Sino
        reprobados <- reprobados + 1;
    FinSi
FinPara
Escribir
"=====
=====";

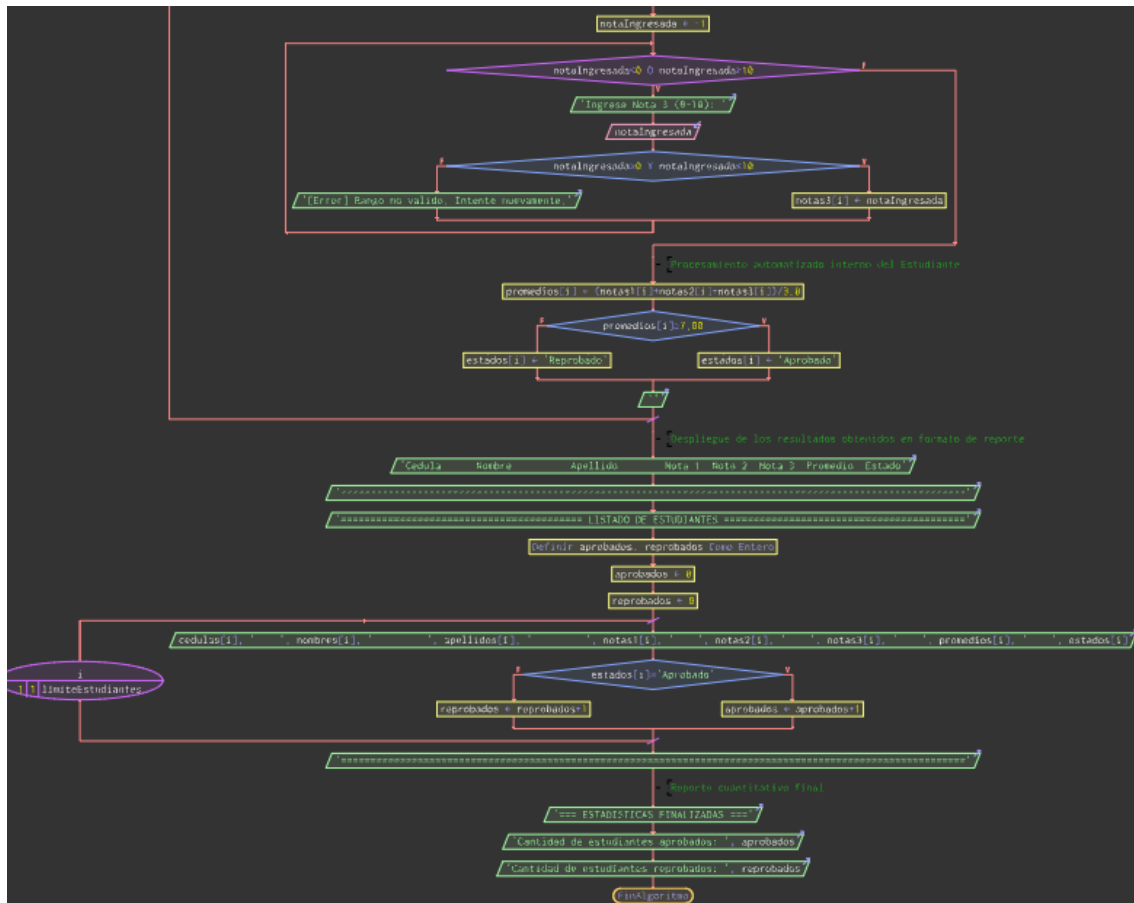
// Reporte cuantitativo final
Escribir "=== ESTADISTICAS FINALIZADAS ===";
Escribir "Cantidad de estudiantes aprobados: ", aprobados;
Escribir "Cantidad de estudiantes reprobados: ", reprobados;
FinAlgoritmo
```

5. Diagrama de Flujo



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026





6. Código en C++

```
#include <iostream>
#include <vector>
#include <string>
#include <iomanip>
#include <cctype>
```

```
using namespace std;
```

```
// CLASE ESTUDIANTE
```

```
class Estudiante {
```

```
private:
```

```
    // Atributos privados para asegurar el encapsulamiento de los datos
```

```
    string cedula;
```

```
    string nombre;
```

```
    string apellido;
```

```
    double nota1;
```

```
    double nota2;
```

```
    double nota3;
```

```
    double promedio;
```

```
    string estado;
```

```
public:
```

```
    // Constructor por defecto para inicializar valores limpios
```

```
    Estudiante() {
```

```
        this->nota1 = 0.0;
```

```
        this->nota2 = 0.0;
```

```
        this->nota3 = 0.0;
```



```
this->promedio = 0.0;
this->estado = "Reprobado";
}

// Constructor parametrizado para la carga directa de datos
Estudiante(string cedula, string nombre, string apellido, double n1, double n2, double n3) {
    this->cedula = cedula;
    this->nombre = nombre;
    this->apellido = apellido;
    this->nota1 = n1;
    this->nota2 = n2;
    this->nota3 = n3;
    // Los cálculos de rendimiento se ejecutan inmediatamente al instanciar
    calcularPromedio();
    determinarEstado();
}

// MÉTODOS GETTERS Y SETTERS
string getCedula() const { return cedula; }
void setCedula(string cedula) { this->cedula = cedula; }

string getNombre() const { return nombre; }
void setNombre(string nombre) { this->nombre = nombre; }

string getApellido() const { return apellido; }
void setApellido(string apellido) { this->apellido = apellido; }

// Si se modifica una nota de forma externa, el promedio y el estado se recalculan al instante
double getNota1() const { return nota1; }
void setNota1(double n1) { this->nota1 = n1; calcularPromedio(); determinarEstado(); }

double getNota2() const { return nota2; }
void setNota2(double n2) { this->nota2 = n2; calcularPromedio(); determinarEstado(); }

double getNota3() const { return nota3; }
void setNota3(double n3) { this->nota3 = n3; calcularPromedio(); determinarEstado(); }

double getPromedio() const { return promedio; }
string getEstado() const { return estado; }

// Realiza el cálculo del promedio aritmético de las tres notas ingresadas
void calcularPromedio() {
    this->promedio = (this->nota1 + this->nota2 + this->nota3) / 3.0;
}

// Evalúa la condición final del alumno basándose en el puntaje de corte (7.00)
void determinarEstado() {
    if (this->promedio >= 7.00) {
        this->estado = "Aprobado";
    } else {
        this->estado = "Reprobado";
    }
}

// Envía a la consola la fila de datos formateada y alineada en columnas
```




UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
void mostrarInformacion() const {
    cout << left << setw(12) << cedula
        << setw(15) << nombre
        << setw(15) << apellido
        << setw(8) << fixed << setprecision(2) << nota1
        << setw(8) << nota2
        << setw(8) << nota3
        << setw(10) << promedio
        << setw(12) << estado << endl;
}
};

// MÉTODOS AUXILIARES DE VALIDACIÓN

/**
 * Filtra la entrada de la cédula asegurando que contenga un formato numérico
 * válido de 10 dígitos y correspondencia regional correcta (Ecuador).
 */
string ingresarCedulaValidada() {
    string ced;
    while (true) {
        cout << "Ingrese Cedula: ";
        cin >> ced;

        // Valida que tenga exactamente 10 caracteres
        if (ced.length() != 10) {
            cout << "[Error] Formato incorrecto. Ingrese exactamente 10 numeros.\n";
            continue;
        }

        // Valida que todos los caracteres ingresados sean numéricos
        bool esNumerico = true;
        for (char c : ced) {
            if (!isdigit(c)) {
                esNumerico = false;
                break;
            }
        }

        if (!esNumerico) {
            cout << "[Error] Formato incorrecto. Ingrese exactamente 10 numeros.\n";
            continue;
        }

        // Comprobación del código de provincia (primeros dos números)
        // stoi convierte el sub-string de los 2 primeros caracteres a entero
        int provincia = stoi(ced.substr(0, 2));
        if ((provincia >= 1 && provincia <= 24) || provincia == 30) {
            return ced;
        } else {
            cout << "[Error] Region o provincia no identificada en el territorio nacional.\n";
        }
    }
}
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
/**
 * Restringe el flujo de captura de calificaciones para mantenerlas
 * dentro de la escala numérica de 0.00 a 10.00 puntos.
 */
double ingresarNotaValidada(string mensaje) {
    double nota;
    while (true) {
        cout << mensaje;
        // Si la entrada es un número válido y está en el rango [0, 10]
        if (cin >> nota && nota >= 0 && nota <= 10) {
            return nota;
        } else {
            cout << "[Error] Rango no valido (Debe ser entre 0 y 10). Intente nuevamente.\n";
            cin.clear(); // Limpia los flags de error del flujo cin
            cin.ignore(10000, '\n'); // Descarta caracteres incorrectos del buffer (como letras)
        }
    }
}

// =====
// CONTROLADOR PRINCIPAL
// =====

int main() {
    vector<Estudiante> listaEstudiantes; // Reemplazo directo de ArrayList en C++
    int limiteEstudiantes = 5;

    cout << "=== SISTEMA DE CONTROL DE ESTUDIANTES Y CALIFICACIONES ===\n";
    cout << "Asignatura: Algoritmos y Logica de Programacion\n\n";

    // Captura estructurada de datos por consola
    for (int i = 0; i < limiteEstudiantes; i++) {
        cout << "--- Registro del Estudiante " << (i + 1) << " ---\n";

        string cedula = ingresarCedulaValidada();

        string nombre, apellido;
        cout << "Ingrese Nombre: ";
        cin >> nombre;
        cout << "Ingrese Apellido: ";
        cin >> apellido;

        double n1 = ingresarNotaValidada("Ingrese Nota 1: ");
        double n2 = ingresarNotaValidada("Ingrese Nota 2: ");
        double n3 = ingresarNotaValidada("Ingrese Nota 3: ");
        cout << endl;

        // Construcción del objeto y almacenamiento en el vector dinámico
        Estudiante estudiante(cedula, nombre, apellido, n1, n2, n3);
        listaEstudiantes.push_back(estudiante);
    }

    // Impresión del reporte general resumido
    cout << "===== LISTADO DE ESTUDIANTES =====\n";
    cout << left << setw(12) << "Cedula"
```



```
<< setw(15) << "Nombre"
<< setw(15) << "Apellido"
<< setw(8) << "Nota 1"
<< setw(8) << "Nota 2"
<< setw(8) << "Nota 3"
<< setw(10) << "Promedio"
<< setw(12) << "Estado" << endl;
cout << "-----\n";

int acumuladorAprobados = 0;
int acumuladorReprobados = 0;

// Recorrido de los objetos para la extracción de métricas cuantitativas
for (const auto& est : listaEstudiantes) {
    est.mostrarInformacion();
    if (est.getEstado() == "Aprobado") {
        acumuladorAprobados++;
    } else {
        acumuladorReprobados++;
    }
}
cout << "=====
=====";

// Bloque informativo final de estadísticas
cout << "=== ESTADISTICAS FINALIZADAS ===";
cout << "Cantidad de estudiantes aprobados: " << acumuladorAprobados << endl;
cout << "Cantidad de estudiantes reprobados: " << acumuladorReprobados << endl;

return 0;
}
```

6.1 Código en Java

```
import java.util.ArrayList;
import java.util.Scanner;
```

```
//Clase Main
```

```
public class Main {
```

```
    // CLASE ESTUDIANTE
```

```
    public static class Estudiante {
```

```
        // Atributos privados para asegurar el encapsulamiento de los datos
```

```
        private String cedula;
```

```
        private String nombre;
```

```
        private String apellido;
```

```
        private double nota1;
```

```
        private double nota2;
```

```
        private double nota3;
```

```
        private double promedio;
```

```
        private String estado;
```

```
        // Constructor por defecto para inicializar valores limpios
```

```
        public Estudiante() {
```

```
            this.nota1 = 0.0;
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
this.nota2 = 0.0;
this.nota3 = 0.0;
this.promedio = 0.0;
this.estado = "Reprobado";
}

// Constructor parametrizado para la carga directa de datos
public Estudiante(String cedula, String nombre, String apellido, double n1, double n2,
double n3) {
    this.cedula = cedula;
    this.nombre = nombre;
    this.apellido = apellido;
    this.nota1 = n1;
    this.nota2 = n2;
    this.nota3 = n3;
    // Los cálculos de rendimiento se ejecutan inmediatamente al instanciar
    calcularPromedio();
    determinarEstado();
}

// MÉTODOS GETTERS Y SETTERS
public String getCedula() { return cedula; }
public void setCedula(String cedula) { this.cedula = cedula; }

public String getNombre() { return nombre; }
public void setNombre(String nombre) { this.nombre = nombre; }

public String getApellido() { return apellido; }
public void setApellido(String apellido) { this.apellido = apellido; }

// Si se modifica una nota de forma externa, el promedio y el estado se recalculan al instante
public double getNota1() { return nota1; }
public void setNota1(double n1) { this.nota1 = n1; calcularPromedio(); determinarEstado();
}

public double getNota2() { return nota2; }
public void setNota2(double n2) { this.nota2 = n2; calcularPromedio(); determinarEstado();
}

public double getNota3() { return nota3; }
public void setNota3(double n3) { this.nota3 = n3; calcularPromedio(); determinarEstado();
}

public double getPromedio() { return promedio; }
public String getEstado() { return estado; }

// Realiza el cálculo del promedio aritmético de las tres notas ingresadas
public void calcularPromedio() {
    this.promedio = (this.nota1 + this.nota2 + this.nota3) / 3.0;
}

// Evalúa la condición final del alumno basándose en el puntaje de corte (7.00)
public void determinarEstado() {
    if (this.promedio >= 7.00) {
        this.estado = "Aprobado";
    }
}
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
} else {
    this.estado = "Reprobado";
}
}

// Envía a la consola la fila de datos formateada y alineada en columnas
public void mostrarInformacion() {
    System.out.printf("%-12s %-15s %-15s %-8.2f %-8.2f %-8.2f %-10.2f %-12s\n",
        cedula, nombre, apellido, nota1, nota2, nota3, promedio, estado);
}
}

// MÉTODOS AUXILIARES DE VALIDACIÓN

private static String ingresarCedulaValidada(Scanner scanner) {
    String ced;
    while (true) {
        System.out.print("Ingrese Cedula: ");
        ced = scanner.next();

        // Valida los 10 dígitos obligatorios usando expresiones regulares
        if (ced.matches("\\d{10}")) {

            int provincia = Integer.parseInt(ced.substring(0, 2));
            if ((provincia >= 1 && provincia <= 24) || provincia == 30) {
                return ced;
            } else {
                System.out.println("[Error] Región o provincia no identificada en el territorio
nacional.");
            }
        } else {
            System.out.println("[Error] Formato incorrecto. Ingrese exactamente 10 numeros.");
        }
    }
}

private static double ingresarNotaValidada(Scanner scanner, String mensaje) {
    double nota;
    while (true) {
        System.out.print(mensaje);
        if (scanner.hasNextDouble()) {
            nota = scanner.nextDouble();
            if (nota >= 0 && nota <= 10) {
                return nota;
            }
        } else {
            scanner.next(); // Limpieza del buffer del scanner ante ingresos tipo String
        }
        System.out.println("[Error] Rango no válido (Debe ser entre 0 y 10). Intente
nuevamente.");
    }
}

// CONTROLADOR PRINCIPAL
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
    ArrayList<Estudiante> listaEstudiantes = new ArrayList<>();
    int limiteEstudiantes = 5;

    System.out.println("=== SISTEMA DE CONTROL DE ESTUDIANTES Y
    CALIFICACIONES ===");
    System.out.println("Asignatura: Algoritmos y Logica de Programacion\n");

    // Captura estructurada de datos por consola
    for (int i = 0; i < limiteEstudiantes; i++) {
        System.out.println("--- Registro del Estudiante " + (i + 1) + " ---");

        String cedula = ingresarCedulaValidada(scanner);

        System.out.print("Ingrese Nombre: ");
        String nombre = scanner.next();
        System.out.print("Ingrese Apellido: ");
        String apellido = scanner.next();

        double n1 = ingresarNotaValidada(scanner, "Ingrese Nota 1: ");
        double n2 = ingresarNotaValidada(scanner, "Ingrese Nota 2: ");
        double n3 = ingresarNotaValidada(scanner, "Ingrese Nota 3: ");
        System.out.println();

        // Construcción del objeto y almacenamiento en el arreglo dinámico
        Estudiante estudiante = new Estudiante(cedula, nombre, apellido, n1, n2, n3);
        listaEstudiantes.add(estudiante);
    }

    // Impresión del reporte general resumido
    System.out.println("===== LISTADO
    DE ESTUDIANTES =====");
    System.out.printf("%-12s %-15s %-15s %-8s %-8s %-8s %-10s %-12s\n",
        "Cedula", "Nombre", "Apellido", "Nota 1", "Nota 2", "Nota 3", "Promedio", "Estado");
    System.out.println("-----
    -----");

    int acumuladorAprobados = 0;
    int acumuladorReprobados = 0;

    // Recorrido de los objetos para la extracción de métricas cuantitativas
    for (Estudiante est : listaEstudiantes) {
        est.mostrarInformacion();
        if (est.getEstado().equals("Aprobado")) {
            acumuladorAprobados++;
        } else {
            acumuladorReprobados++;
        }
    }

    System.out.println("=====
    =====");

    // Bloque informativo final de estadísticas
```



```
System.out.println("=== ESTADISTICAS FINALIZADAS ===");
System.out.println("Cantidad de estudiantes aprobados: " + acumuladorAprobados);
System.out.println("Cantidad de estudiantes reprobados: " + acumuladorReprobados);

scanner.close();
}
}
```

1.7 Resultados obtenidos

Se logró la construcción exitosa de dos aplicaciones de consola funcionales y equivalentes en los lenguajes C++ y Java. Ambos programas ejecutan de manera automatizada el cálculo del promedio aritmético simple y evalúan de forma inmediata la condición de aprobación.

El comportamiento modular del sistema garantiza que cualquier modificación en las notas individuales recalculen el estado de manera síncrona mediante la lógica interna de los métodos calcularPromedio() y determinarEstado().

1.8 Habilidades blandas empleadas en la práctica

- ☒ Liderazgo
- ☐ Trabajo en equipo
- ☐ Comunicación asertiva
- ☐ La empatía
- ☒ Pensamiento crítico
- ☒ Flexibilidad
- ☐ La resolución de conflictos
- ☒ Adaptabilidad
- ☒ Responsabilidad

1.9 Conclusiones

- La Programación Orientada a Objetos demostró ser un paradigma altamente eficiente para modelar entidades del mundo real, permitiendo que la abstracción de un Estudiante contenga tanto sus datos como la cédula, nombre, notas como el comportamiento analítico indispensable para su gestión.
- El uso de funciones de validación robustas mitiga el riesgo de corrupción de datos operativos por errores humanos en el ingreso por teclado, garantizando la estabilidad del software ante tipos de datos incompatibles o valores fuera de los límites definidos.
- La unificación de criterios lógicos aplicados sobre tecnologías distintas (C++ y Java) ratifica que las bases de la lógica de programación y el diseño arquitectónico de software trascienden las particularidades sintácticas de un lenguaje específico.

1.10 Recomendaciones

- Practicar más la Programación Orientada a Objetos (POO) tanto en C++ como en Java, realizando ejercicios similares para dominar por completo el uso de clases, objetos y métodos.
- Probar el programa con más de 5 estudiantes y con diferentes tipos de datos para asegurar que el sistema no falle y que las validaciones de las notas funcionen en cualquier situación.
- Seguir comentando el código de forma clara y utilizar nombres de variables fáciles de entender, lo cual ayuda mucho a mantener el orden dentro del repositorio del proyecto.



- Añadir una opción para guardar los datos, de modo que la información de los estudiantes no se borre al cerrar la consola y se pueda revisar después.

1.11 Referencias bibliográficas

- [1] F. J. Ceballos, *Programación orientada a objetos con C++*, 5ta ed. Madrid, España: Editorial RA-MA, 2015.
- [2] P. Deitel y H. Deitel, *Java: Cómo programar*, 10ma ed. Ciudad de México, México: Pearson Educación, 2016.
- [3] H. Schildt, *C++: The complete reference*, 5ta ed. Nueva York, EE. UU.: McGraw-Hill Education, 2019.

1.12 Anexos

```
--- Registro del Estudiante 1 ---
Ingrese Cedula: 1804834248
Ingrese Nombre: Mateo
Ingrese Apellido: Muyulema
Ingrese Nota 1: 7
Ingrese Nota 2: 10
Ingrese Nota 3: 8

--- Registro del Estudiante 2 ---
Ingrese Cedula: 123456
[Error] Formato incorrecto. Ingrese exactamente 10 numeros.
Ingrese Cedula: 1234567890
Ingrese Nombre: Jose
Ingrese Apellido: Urbina
Ingrese Nota 1: 2
Ingrese Nota 2: 4
Ingrese Nota 3: 5

--- Registro del Estudiante 3 ---
Ingrese Cedula: 1324354647
Ingrese Nombre: Sofia
Ingrese Apellido: Reyes
Ingrese Nota 1: 4
Ingrese Nota 2: 7
Ingrese Nota 3: 9

--- Registro del Estudiante 4 ---
Ingrese Cedula: 1987654312
Ingrese Nombre: Claudia
Ingrese Apellido: Martinez
Ingrese Nota 1: 5
Ingrese Nota 2: 6
Ingrese Nota 3: 10

--- Registro del Estudiante 5 ---
Ingrese Cedula: |
```




UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



--- Registro del Estudiante 5 ---
Ingreso Cedula: 1346789085
Ingreso Nombre: Jhair
Ingreso Apellido: Monar
Ingreso Nota 1: 10
Ingreso Nota 2: 8
Ingreso Nota 3: 9

LISTADO DE ESTUDIANTES							
Cedula	Nombre	Apellido	Nota 1	Nota 2	Nota 3	Promedio	Estado
1804834248	Mateo	Muyulema	7.00	10.00	8.00	8.33	Aprobado
1234567890	Jose	Urbina	2.00	4.00	5.00	3.67	Reprobado
1324354647	Sofia	Reyes	4.00	7.00	9.00	6.67	Reprobado
1987654312	Claudia	Martinez	5.00	6.00	10.00	7.00	Aprobado
1346789085	Jhair	Monar	10.00	8.00	9.00	9.00	Aprobado

=== ESTADISTICAS FINALIZADAS ===
Cantidad de estudiantes aprobados: 3
Cantidad de estudiantes reprobados: 2

Ejecución del ejercicio en Java

```
--- Registro del Estudiante 1 ---
Ingreso Cedula: 1804834248
Ingreso Nombre: Antonela
Ingreso Apellido: Narvaez
Ingreso Nota 1: 8
Ingreso Nota 2: 7
Ingreso Nota 3: 10

--- Registro del Estudiante 2 ---
Ingreso Cedula: 1804567382
Ingreso Nombre: Jhair
Ingreso Apellido: Monar
Ingreso Nota 1: 5
Ingreso Nota 2: 7
Ingreso Nota 3: 10

--- Registro del Estudiante 3 ---
Ingreso Cedula: 2345
[Error] Formato incorrecto. Ingrese exactamente 10 numeros.
Ingreso Cedula: 1804567892
Ingreso Nombre: Sebastian
Ingreso Apellido: Guanga
Ingreso Nota 1: 5
Ingreso Nota 2: 2
Ingreso Nota 3: 4

--- Registro del Estudiante 4 ---
Ingreso Cedula: 1804231456
Ingreso Nombre: Josue
Ingreso Apellido: Atiencia
Ingreso Nota 1: 5
Ingreso Nota 2: 10
Ingreso Nota 3: 3

--- Registro del Estudiante 5 ---
Ingreso Cedula: 1804658392
Ingreso Nombre: Alejandro
Ingreso Apellido: Villacres
Ingreso Nota 1: 3
Ingreso Nota 2: 7
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
===== LISTADO DE ESTUDIANTES =====
Cedula      Nombre      Apellido      Nota 1  Nota 2  Nota 3  Promedio  Estado
-----
1804834248  Antonela    Narvaez       8.00    7.00    10.00    8.33     Aprobado
1804567382  Jhair       Monar         5.00    7.00    10.00    7.33     Aprobado
1804567892  Sebastian   Guanga        5.00    2.00    4.00     3.67     Reprobado
1804231456  Josue       Atiencia      5.00    10.00   3.00     6.00     Reprobado
1804658392  Alejandro   Villacres     3.00    7.00    10.00    6.67     Reprobado
=====

=== ESTADISTICAS FINALIZADAS ===
Cantidad de estudiantes aprobados: 2
Cantidad de estudiantes reprobados: 3

Process returned 0 (0x0)   execution time : 118.373 s
Press any key to continue.
```

Ejecución del ejercicio en C++